

Solution to Longest repeated Subsequence Length problem

```
public class Exercise {
    static int
    findLongestRepeatingSubSeq(String str)
    {
        int n = str.length();
        int[][] dp = new int[n+1][n+1];

        for (int i=1; i<=n; i++)
        {
            for (int j=1; j<=n; j++)
            {
                if (str.charAt(i-1) ==
str.charAt(j-1) && i!=j)
                    dp[i][j] = 1 +
dp[i-1][j-1];
                else
                    dp[i][j] =
Math.max(dp[i][j-1], dp[i-1][j]);
            }
        }
        return dp[n][n];
    }
}
```

Solution to Longest Common Subsequence Length problem using Top Down Approach

```
public class Exercise {
    public int findLCSLengthTD(String s1,
String s2) {
        int[][] dp = new
int[s1.length()][s2.length()]; //dp table will
store all the results
```

```

        for (int i = 0; i < s1.length(); i++)
// initialize all values with '-1'
            for (int j = 0; j < s2.length();
j++)
                dp[i][j] = -1;
        return findLCSLengthAux(dp, s1, s2, 0,
0);
    } //end of method

```

```

    private int findLCSLengthAux(int[][] dp,
String s1, String s2, int i1, int i2) {
        if (i1 == s1.length() || i2 ==
s2.length()) //Base case
            return 0;

        if (dp[i1][i2] == -1) { //if we have
not already solved this problem, only then
solve it
            if (s1.charAt(i1) ==
s2.charAt(i2)) {
                dp[i1][i2] = 1 +
findLCSLengthAux(dp, s1, s2, i1 + 1, i2 + 1);
//If current character in both the string
matches, then increase the index by 1 in both
the strings.

                return dp[i1][i2];
            } else {
                int c1 = findLCSLengthAux(dp,
s1, s2, i1, i2 + 1); //Increase index of 2nd
String

                int c2 = findLCSLengthAux(dp,
s1, s2, i1 + 1, i2); //Increase index of 1st
String

                dp[i1][i2] = Math.max(c1, c2);

```

```

        }
    }
    return dp[i1][i2];
} //end of method

}

```

Solution to Longest Common Subsequence Length problem using Bottom Up Approach

```

public class Exercise {
    public int findLCSLengthBU(String s1,
String s2) {

        int[][] dp = new int[s1.length() +
1][s2.length() + 1];
        for (int i = s1.length(); i >= 1; i--)
        {

            for (int j = s2.length(); j >= 1;
j--) {

                if (s1.charAt(i-1) ==
s2.charAt(j-1)) {
                    dp[i-1][j-1] =
Math.max((1 + dp[i][j]), Math.max(dp[i][j-1],
dp[i-1][j]));
                }
                else {
                    dp[i-1][j-1] =
Math.max(dp[i][j-1], dp[i-1][j]);
                }
            } //end of inner loop
        } //end of loop
        return dp[0][0];
    } //end of method
}

```

```
}
```

Solution to Longest Common Subsequence

```
public class Exercise {
    static char[] lcs(String X, String Y, int
m, int n)
    {
        int[][] L = new int[m+1][n+1];

        // Following steps build L[m+1][n+1]
in bottom up fashion. Note
        // that L[i][j] contains length of LCS
of X[0..i-1] and Y[0..j-1]
        for (int i=0; i<=m; i++)
        {
            for (int j=0; j<=n; j++)
            {
                if (i == 0 || j == 0)
                    L[i][j] = 0;
                else if (X.charAt(i-1) ==
Y.charAt(j-1))
                    L[i][j] = L[i-1][j-1] + 1;
                else
                    L[i][j] =
Math.max(L[i-1][j], L[i][j-1]);
            }
        }

        // Following code is used to print LCS
        int index = L[m][n];
        int temp = index;
```

```

        // Create a character array to store
the lcs string
        char[] lcs = new char[index+1];
        lcs[index] = '\u0000'; // Set the
terminating character

        // Start from the
right-most-bottom-most corner and
        // one by one store characters in
lcs[]

        int i = m;
        int j = n;
        while (i > 0 && j > 0)
        {
            // If current character in X[] and
Y are same, then
            // current character is part of
LCS
            if (X.charAt(i-1) ==
Y.charAt(j-1))
            {
                // Put current character in
result

                lcs[index-1] = X.charAt(i-1);

                // reduce values of i, j and
index

                i--;
                j--;
                index--;
            }

            // If not same, then find the
larger of two and

```

```

        // go in the direction of larger
value
        else if (L[i-1][j] > L[i][j-1])
            i--;
        else
            j--;
    }

    // Print the lcs

    return lcs;

}

}

```

Solution to Longest Palindromic Subsequence using Top Down Approach

```

public class Exercise {
    public int findLPSSLength(String string) {
        Integer[][] dp = new
Integer[string.length()][string.length()];
        return lps_Aux(dp, string, 0,
string.length() - 1);
    } // end of method

    private int lps_Aux(Integer[][] dp, String
string, int startIndex, int endIndex) {
        if (startIndex > endIndex) { //BASE
CASE - If we have traversed more than 1/2 of

```

```

string then return 0 as we dont need to
process it
        return 0;
    }
    if (startIndex == endIndex) { //BASE
CASE - If both the index are at same position
then its a palindrome as its 1 character.
        return 1;
    }
    int c3 = 0;
    if (dp[startIndex][endIndex] == null)
{ //If we have not solved this problem
previously, only then solve it
        if (string.charAt(startIndex) ==
string.charAt(endIndex)) {
            c3 = 2 + lps_Aux(dp, string,
startIndex + 1, endIndex - 1); //CASE#1 - If
index pointed characters matches then we add 2
to the existing known palindrome length
        }
        int c1 = lps_Aux(dp, string,
startIndex + 1, endIndex); //CASE#2 - Skip one
element from beginning
        int c2 = lps_Aux(dp, string,
startIndex, endIndex - 1); //CASE#3 - Skip one
element from end
        dp[startIndex][endIndex] =
Math.max(c3, Math.max(c1, c2)); //Take the max
sized palindrome
    }
    return dp[startIndex][endIndex];
} // end of method
}

```

Solution to Longest Palindromic Subsequence using Bottom Up Approach

```
public class Exercise {
    public int findLPSLengthBU(String st) {
        int[][] dp = new
int[st.length()][st.length()];
        for (int col = 0; col < st.length();
col++) {
            for (int row = st.length()-1; row
>= 0; row--) {
                if (row > col) { // BASE CASE
- If we have traversed more than 1/2 of string
then return 0 as we dont need to process it
                    dp[row][col] = 0;
                } else if (row == col) { //
BASE CASE - If both the index are at same
position then its a palindrome as its 1
character.
                    dp[row][col] = 1;
                } else {
                    if (st.charAt(row) ==
st.charAt(col)) {
                        dp[row][col] =
Math.max(2+dp[row + 1][col - 1],
Math.max(dp[row][col - 1], dp[row + 1][col]));
                    } else {
                        dp[row][col] =
Math.max(dp[row][col - 1], dp[row + 1][col]);
                    }
                }
            }
        }
        return dp[0][st.length()-1];
    }
}
```



```
}
```

Solution to Longest Palindromic Substring

```
public class Exercise {
    public String longestPalindrome(String s) {
        int length = s.length();
        String res = s.substring(0, 1);
        for(int i = 0; i < length; i++) {
            // base case (i, i) and (i, i + 1);
            String ps = getPalindrome(s, i, i);
            res = (ps.length() > res.length())
? ps : res;
            ps = getPalindrome(s, i, i + 1);
            res = (ps.length() > res.length())
? ps : res;
        }
        return res;
    }
    // expand around center
    private String getPalindrome(String s, int
l, int r){
        while(l >= 0 && r < s.length() &&
s.charAt(l) == s.charAt(r)){
            l--;
            r++;
        }
        return s.substring(l + 1, r);
    }
}
```